Technical Assessment: Senior Full-Stack Engineer

Building an Embeddable Dynamic Workflow & Form Rendering Engine

Target Role: Lead Full-Stack Developer | **Timeframe:** 7 Days

1 Executive Summary & Objective

We are seeking a high-caliber Full-Stack Engineer who can bridge the gap between elegant client-facing interfaces, secure multi-tenant routing, and modern cloud infrastructure.

The goal of this assignment is to design and prototype a highly scalable, **"Stripe-like" embeddable workflow platform** designed for law firms. The platform must empower lawyers to build customizable intake forms or automated service components (e.g., entity registration or contract upload workflows) on an admin portal, instantly embed them into their personal websites, and execute them securely.

2 End-to-End Platform Flow Diagram

The following diagram illustrates the complete developer/user lifecycle: from creating and designing the custom legal workflow inside our platform, to embedding it, sync propagation, and secure runtime submission.

Step 1: Initialize Workflow
Lawyer accesses the central Next.js Admin Dashboard and creates a new workflow service (e.g., “Company Formation”).

Step 2: Design Form & Inputs
Lawyer adds, reorders, and configures field schemas (Text, Select options, File upload slots, Dynamic lists).

Step 3: Theme Customization (Real-Time Visuals)
Lawyer modifies style configurations (background colors, fonts, input border-radius, button styles, alignment).

Step 4: Save & Map Webhook Destination
System persists schema and visual JSON blocks to Firestore, and maps incoming entries to external law firm webhooks.

Step 5: Client Site Integration
Lawyer copies a lightweight JavaScript snippet and pastes it directly onto their personal firm website.

Step 6: Dynamic Runtime Load & Real-Time Sync
Script triggers on load, fetches the layout config dynamically, and builds the UI. **No manual redeployment needed on save.**

Step 7: HMAC-Signed Ingestion & Core Security
Submission triggers. Server executes Origin/Token verification, writes data to Firestore, and sends signed webhooks out.

3 The Request & Validation Cycle

To safeguard user environments and keep multi-tenant pipelines safe from data tampering, the platform processes events along the following verification loop:

1. **Widget Call:** Embedded client script calls Next.js app servers (GCP Cloud Run) requesting visual assets and layout definitions.
2. **Origins Audit:** API verifies the parent container's context origin headers against registered allowed domains in Cloud Firestore.
3. **Schema Injection:** Verified configurations load, programmatically populating input items, button designs, and layouts without compiling dedicated page versions.
4. **Payload Ingestion:** Secure endpoints receive transaction schemas and apply script sanitization filters to prevent standard XSS or injection vulnerabilities.
5. **HMAC Handshake:** The platform generates a unique payload signature using a shared HMAC-SHA256 secret. This allows external systems listening to the webhooks to confirm they originate securely from your system.

4 Target Workflow Examples to Implement

Your platform must be flexible enough to handle diverse legal services. Below are two concrete examples you should design for. Your system should be capable of modeling, styling, and processing both configurations:

4.1 Example 1: "Create a Company" (Corporate Formation Form)

- **Target Inputs:**
  - Company Name Preference (Text Input)
  - Entity Type (Dropdown: LLC, C-Corp, S-Corp)
  - State of Incorporation (Dropdown)
  - Shareholder List (Dynamic List of Text & Percentage Inputs)
- **Target UI Customization:** Clean dark slate background, rounded inputs, and compact forms.
- **Action / Webhook Target:** Emits structured JSON payloads containing the formation metadata directly to a corporate registry microservice.

4.2 Example 2: "Review Contract" (Intake & Document Gateway)

- **Target Inputs:**
  - Client Name & Corporate Email (Validated fields)
  - Contract Type (Dropdown: NDA, SaaS Agreement, Employment contract)
  - Urgency Level (Radio buttons: Low, Medium, High)
  - PDF Agreement Upload (File Upload field)

- **Target UI Customization:** Formal corporate layout, warning labels for high urgency, and large drop-zone file upload buttons.
- **Action / Webhook Target:** Forwards the document and metadata to an AI triage parsing service to trigger automated redlining.

5 Detailed Requirements

5.1 Part A: No-Code Builder (Next.js Admin Portal)

- **Visual Form Constructor:** Click-to-add or drag-and-drop builder interface where a lawyer can construct input sequences (Text, Dynamic lists, File uploads).
- **Live Theme Customizer:** Real-time visual style configurer (colors, spacing, fonts, button styling, corner rounding).
- **Firestore Schema Persistence:** Store the configurations, layouts, and backend webhook destinations in a clean, query-optimized Firestore layout.

5.2 Part B: Dynamic Embedded Runtime & Cryptographic Security

- **Dynamic Widget Rendering Engine:** Provide a single client-side embeddable script (or custom script tag) that lawyers paste onto their own websites. This script must dynamically generate the correct UI components, structural layouts, and CSS styling on-the-fly based on the fetched configuration.
- **Instant Sync / Zero-Rebuild propagation:** When a lawyer alters a form workflow's inputs, themes, or colors in the Admin Dashboard and saves it, those changes must instantly take effect on the lawyer's external website on the very next render/call of the script without any manual redeployment.
- **Validation Layer:** Endpoints processing submissions must prevent injection attacks.
- **Origin Checks:** The API must check the incoming headers against the lawyer's registered domain in Firestore.
- **Webhook Integrity:** Sign outgoing payloads using an HMAC-SHA256 signature calculated with a secret key shared with the lawyer, allowing client backends to confirm requests originate genuinely from your system.

5.3 Part C: DevOps & GCP Automation

- Provide a Terraform configuration or clean deployment instructions to ship a single-container Next.js application to **GCP Cloud Run**.
- Detail how the central platform is structured to serve the dynamic UI bundles and handle API submissions efficiently under a single unified domain or load balancer context.

6 Feature Prioritization Matrix

We value deep, production-ready, and clean implementations of high-priority items over incomplete implementations of the entire list. Please focus your development in this sequence:

Priority	Feature Domain	Success Criteria
1	Visual Design Interface	Lawyers can customize forms, buttons, colors, and preview in real time.
2	Dynamic Client Sync	Embedding the script on an external site renders UI and reflects changes instantly on save.
3	Workflow Schema Engine	Support for dynamic inputs (e.g. "Create a Company" list requirements).
4	Backend Core & Webhooks	Secure ingestion API routes that write to Firestore and fire webhooks.
5	Domain & Token Guard	Implementation of secure Origin Validation and HMAC signature generation.
6	IaC & Cloud Run Setup	Fully functional Terraform codebase deploying components to GCP.

7 Bonus Challenge (Optional)

AI-Generated Layouts: Provide an integration with an LLM (e.g., OpenAI, Anthropic) where a lawyer can write: *"Give me a friendly, light-themed slip-and-fall intake flow,"* and the tool auto-generates the input fields, colors, and validations automatically.

8 Submission Instructions

1. **Repository Setup:** Host your code in a Git-based repository (GitHub or GitLab). Ensure a clean, logical commit history.
2. **README Documentation:**
   - Detail step-by-step local development setup instructions.
   - Explain the security protocol design (HMAC signing details).
   - Explain how the dynamic script fetches config on client-load.
   - Highlight GCP and Terraform architectures.
3. **Presentation/Demo:** We will conduct a 30-minute system walkthrough where you will present the builder interface, simulate an embed submit flow, and explain your security checks.